Exercício em duplas: qual é a saída? (1/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual conceito da aula explica?

1 · Python

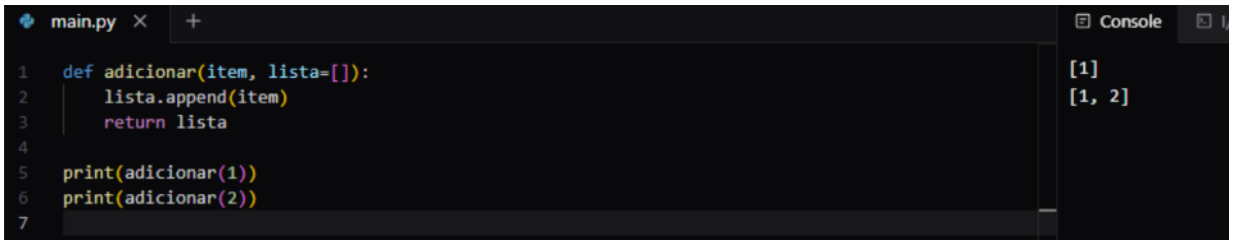
```python
def adicionar(item, lista=[]):
    lista.append(item)
    return lista

print(adicionar(1))
print(adicionar(2))
```

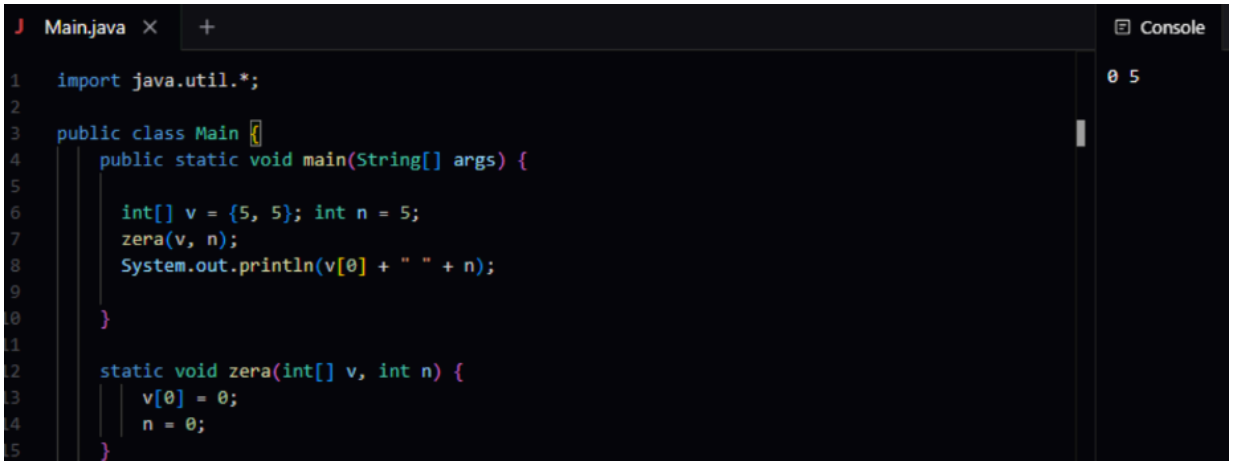
2 · Java

```java
static void zera(int[] v, int n) {
    v[0] = 0;
    n = 0;
}
int[] v = {5, 5}; int n = 5;
zera(v, n);
System.out.println(v[0] + " " + n);
```

Python:



Java:



Python: a saída é `[1]` e depois `[1, 2]`. O valor padrão `lista=[]` é avaliado uma única vez, na definição da função, e a mesma lista é reaproveitada em todas as chamadas que não passam o argumento. É um dos bugs mais famosos da linguagem. Para corrigir, use `lista=None` e, dentro da função, `if lista is None: lista = []`. Conceitos: parâmetros padrão (p. 368) e passagem por atribuição (p. 384).

Java: a saída é `0 5`. O array é um objeto, e o que vai para o método é uma cópia da

referência, então alterar `v[0]` dentro dele afeta o array original. Já o `int` é copiado por valor, e mudar a cópia não altera a variável de fora. Conceito: Java passa tudo por valor (p. 383).

Exercício: qual é a saída? (2/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual conceito da aula explica?

3 · Python

```python
fs = [lambda: i for i in range(3)]
print([f() for f in fs])
```

4 · C

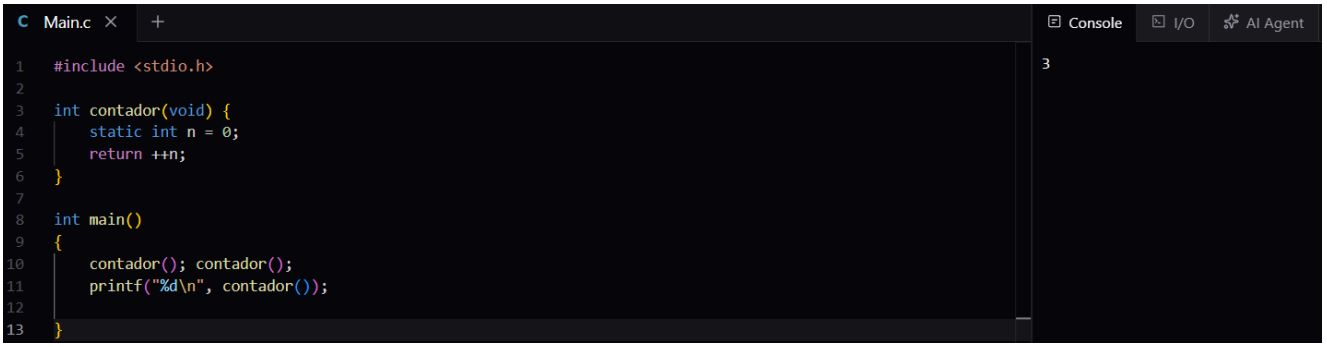
```c
int contador(void) {
    static int n = 0;
    return ++n;
}
// em main:
contador(); contador();
printf("%d\n", contador());
```

SAÍDAS:

3 · Python



4 · C



Python: a saída é `[2, 2, 2]`. As lambdas não guardam o valor de `i`, e sim a própria variável. Quando são chamadas, o loop já terminou e `i` vale 2 (late binding). Para corrigir, use `lambda i=i: i`, que fixa o valor atual como parâmetro padrão no momento da criação. Conceito: fechamentos (p. 404), o mesmo problema clássico do `var` em JS.

C: a saída é `3`. A variável `static` é inicializada uma única vez e mantém o valor entre as chamadas, então cada chamada continua de onde a anterior parou. Conceito: variáveis locais

estáticas (p. 373-374).

Exercício: qual é a saída? (3/3)

Para cada trecho: **(a)** qual é a saída? **(b)** qual conceito da aula explica?

5 · Rust

```rust
fn dobra(v: Vec<i32>) -> Vec<i32> {
    v.iter().map(|x| x * 2).collect()
}
let v = vec![1, 2, 3];
let d = dobra(v);
println!("{:?} {:?}", v, d);
```

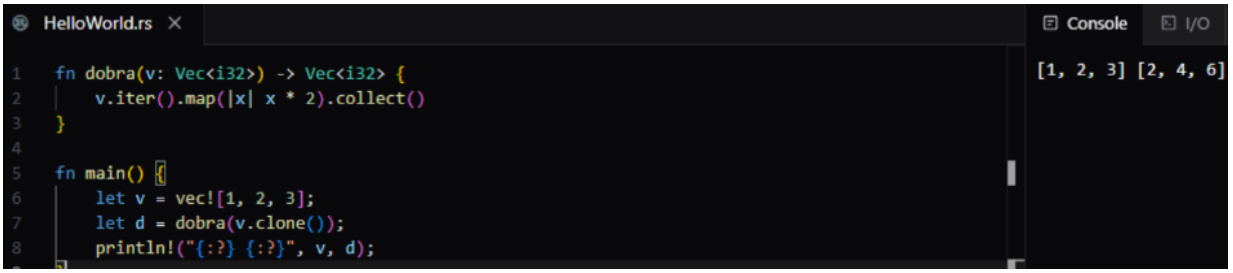
6 · Python

```python
total = 0

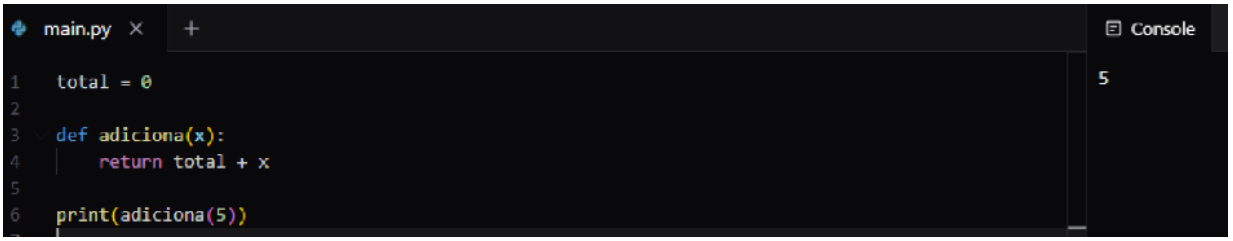
def adiciona(x):
    total = total + x
    return total

print(adiciona(5))
```

Rust



Python



Rust: não compila (erro E0382). O vetor `v` foi movido para dentro de `dobra`, que passou a ser dona dele, então ele não pode mais ser usado depois da chamada. Para resolver, mude a assinatura para `fn dobra(v: &Vec<i32>)` e chame com `dobra(&v)`. Conceito: modos de passagem de parâmetros em Rust (posse e empréstimo).

Python: `UnboundLocalError`. Como `total` recebe uma atribuição dentro da função, o Python já o trata como variável local na hora de compilar, e a leitura acontece antes de existir qualquer valor nela. Dá pra resolver com `global total`, mas o melhor é receber o valor como parâmetro e devolver o resultado. Conceito: ambiente de referenciamento local (p. 373)

e escopo (aula anterior).